

Manual Técnico de Usuario

Software de Correlación Digital de Imágenes 2D

DIC_2D_V3.1

Digital Image Correlation — 2D Analysis Pipeline

Documento técnico correspondiente al paquete `run_dic_v3.py` /

DIC_2D_V3

Microcredencial — Prueba Piloto

Beca Santander – FIMPES Investigación

Índice general

1	Introducción y alcance del documento	3
2	Fundamentos conceptuales implementados	3
2.1	Hipótesis de conservación de intensidad	3
2.2	Criterio de correlación: ZNCC	3
2.3	Refinamiento subpíxel por interpolación parabólica	4
2.4	Modo de correlación incremental	4
2.5	Cálculo de strain: gradientes de desplazamiento por LSQ local	5
2.6	Propagación de incertidumbre (modo GUM)	5
3	Estructura del paquete	6
4	Parámetros de entrada del script principal	7
4.1	Bloque PROJECT: rutas y nombre del proyecto	7
4.2	Bloque COMPUTE: cómputo y aceleración	8
4.3	Bloque IMAGING: imagen y calibración	9
4.4	Bloque ROI: región de interés y grilla de análisis	10
4.5	Bloque DIC: parámetros del motor de correlación	12
4.6	Bloque DISPLACEMENT: postproceso de desplazamiento	13
4.7	Bloque STRAIN: deformación	14
4.8	Bloque UNCERTAINTY: incertidumbre	14
4.9	Bloque VIS: visualización y exportación	15
4.10	Bloque RUNTIME: control de ejecución	16
5	Descripción técnica de los módulos	16
5.1	Carga de imágenes: image_loader.py	16
5.2	Calibración: calibration.py	17
5.3	ROI y grilla: roi_grid.py	18
5.4	Motor de correlación: correlation_engine.py	18
5.4.1	Recorte de cola de correlación	19
5.5	Postproceso de desplazamiento: displacement.py	20
5.6	Cálculo de strain: strain.py	20
5.7	Incertidumbre: uncertainty.py	20
6	Ejecución del pipeline	21
6.1	Prerrequisitos del sistema	21
6.2	Estructura de directorios esperada	21
6.3	Ejecución estándar	21
7	Interpretación de las salidas	22
7.1	Dashboard global	22
7.2	Campo vectorial de desplazamiento (quiver)	23
7.3	Mapas individuales de deformación	24
7.4	Evolución temporal de la deformación	25

7.5	Strain vs. desplazamiento máximo	26
7.6	Resumen de incertidumbre	26
7.7	Tabla GUM	27
8	Salidas del frame seleccionado	27
9	Archivos generados por el pipeline	30
9.1	Archivos de calibración	30
9.2	Archivos de datos numéricos	30
9.2.1	Strain	30
9.2.2	Incertidumbre	30
9.3	Estado del proyecto	30
10	Criterios de configuración recomendados	30
11	Limitaciones de la versión actual	31

1 Introducción y alcance del documento

Este manual describe el funcionamiento técnico del software **DIC_2D_V3.1**, un pipeline modular en Python para Correlación Digital de Imágenes en dos dimensiones (DIC 2D). El software fue desarrollado en el marco de la Microcredencial de Análisis de Esfuerzos mediante DIC, Prueba Piloto, bajo la Beca Santander–FIMPES Investigación.

El presente documento es complementario al *Manual Técnico del Participante*. Mientras ese manual establece los fundamentos teóricos de DIC (criterios de correlación, subsets, calibración, tensor de deformación, incertidumbre GUM), el presente manual describe *cómo esos conceptos están implementados concretamente* en el código DIC_2D_V3.1, qué parámetros controlan cada etapa y cómo interpretar las salidas numéricas y gráficas generadas.

El orquestador principal es `run_dic_v3.py`. Todos los parámetros de configuración están centralizados al inicio de ese script. El usuario no necesita editar los módulos del paquete DIC_2D_V3/ para una corrida estándar.

2 Fundamentos conceptuales implementados

Vínculo con Manual del Participante

Esta sección resume los conceptos desarrollados con detalle matemático en los Capítulos 1, 3 y 4 del *Manual Técnico del Participante*. Se recomienda consultar ese documento antes de ejecutar el pipeline por primera vez.

2.1 Hipótesis de conservación de intensidad

El principio fundamental de DIC es que la intensidad de la imagen se conserva entre la imagen de referencia $f(\mathbf{x})$ y la imagen deformada $g(\mathbf{x})$ bajo la transformación del subset:

$$f(\mathbf{x}) \approx g(\mathbf{x} + \mathbf{u}(\mathbf{x})), \quad (1)$$

donde $\mathbf{u} = (u, v)$ es el campo de desplazamiento en el plano de la imagen. El objetivo de la correlación es encontrar $\mathbf{u}$ que minimiza la discrepancia entre el subset de referencia y el subset deformado.

2.2 Criterio de correlación: ZNCC

En `correlation_engine.py`, el núcleo de búsqueda utiliza la función `cv2.matchTemplate` con el criterio `TM_CCOEFF_NORMED`, que es numéricamente equivalente al coeficiente de correlación cruzada normalizado con substracción de media (ZNCC, *Zero-Normalized Cross*

Correlation):

$$C_{\text{ZNCC}}(u, v) = \frac{\sum_{(\xi, \eta) \in S} [f(\xi, \eta) - \bar{f}] [g(\xi + u, \eta + v) - \bar{g}_{(u, v)}]}{\sqrt{\sum_{(\xi, \eta) \in S} [f(\xi, \eta) - \bar{f}]^2 \sum_{(\xi, \eta) \in S} [g(\xi + u, \eta + v) - \bar{g}_{(u, v)}]^2}}, \quad (2)$$

donde S es el dominio del subset de radio r (`subset_radius`), $\bar{f}$ y $\bar{g}_{(u, v)}$ son las medias locales de intensidad. Este criterio es invariante a cambios de offset y ganancia de iluminación, lo que lo hace robusto frente a variaciones de iluminación entre frames. El rango de C_{ZNCC} es $[-1, 1]$; el parámetro `corrcoef_min` define el umbral de aceptación mínima.

2.3 Refinamiento subpíxel por interpolación parabólica

Una vez identificado el máximo entero del mapa de correlación en el píxel $(i_{\text{max}}, j_{\text{max}})$, se aplica un ajuste parabólico unidimensional en cada dirección para estimar la posición subpíxel del pico. Para la dirección x :

$$\delta_x = \frac{C(i_{\text{max}} - 1) - C(i_{\text{max}} + 1)}{2[C(i_{\text{max}} - 1) - 2C(i_{\text{max}}) + C(i_{\text{max}} + 1)]}, \quad (3)$$

y análogamente para y . El desplazamiento subpíxel δ_x se añade a la posición entera del máximo. Este esquema está implementado en la función `_parabolic_subpixel_1d()` y corresponde al método de interpolación parabólica del pico descrito en el Manual del Participante (§1.6).

2.4 Modo de correlación incremental

El parámetro `mode = "incremental"` activa el modo de correlación paso a paso (*step analysis*), análogo a la estrategia de seguimiento material de NCorr. En este modo, para cada par de frames consecutivos $(k - 1, k)$:

1. El template se extrae alrededor de la posición *material* del punto al final del frame anterior: $(x_0 + u_{k-1}, y_0 + v_{k-1})$.
2. Se aplica un predictor basado en el incremento previo: la ventana de búsqueda se centra en $(x_0 + u_{k-1} + \Delta u_{k-1}, y_0 + v_{k-1} + \Delta v_{k-1})$.
3. El incremento medido $(\Delta u_k, \Delta v_k)$ se acumula para obtener el desplazamiento total respecto a la referencia: $u_k = u_{k-1} + \Delta u_k$.

Este mecanismo es esencial para ensayos con desplazamientos grandes donde la correlación directa frame–referencia fallaría por pérdida del patrón moteado dentro del subset original.

2.5 Cálculo de strain: gradientes de desplazamiento por LSQ local

El módulo `strain.py` implementa el método `ncorr_dispgrad`, que aproxima el enfoque del *Virtual Strain Gauge* de NCorr. En cada nodo de la grilla (i, j) , se ajusta localmente un plano de primer orden sobre una vecindad circular de radio `strain_radius` r :

$$u(x, y) \approx a_0 + a_1 x + a_2 y, \quad v(x, y) \approx b_0 + b_1 x + b_2 y, \quad (4)$$

resolviendo el sistema de mínimos cuadrados:

$$\min_{\mathbf{a}} \|\mathbf{A}\mathbf{a} - \mathbf{u}_{\text{local}}\|^2, \quad \mathbf{A} = \begin{bmatrix} 1 & x_1 & y_1 \\ \vdots & \vdots & \vdots \\ 1 & x_N & y_N \end{bmatrix}, \quad (5)$$

donde N es el número de nodos dentro de la vecindad circular. Los gradientes de desplazamiento extraídos son $\partial u / \partial x = a_1$, $\partial u / \partial y = a_2$, $\partial v / \partial x = b_1$, $\partial v / \partial y = b_2$.

Para el tensor de deformación infinitesimal (opción activa por defecto):

$$\varepsilon_{xx} = \frac{\partial u}{\partial x}, \quad \varepsilon_{yy} = \frac{\partial v}{\partial y}, \quad \varepsilon_{xy} = \frac{1}{2} \left(\frac{\partial u}{\partial y} + \frac{\partial v}{\partial x} \right). \quad (6)$$

2.6 Propagación de incertidumbre (modo GUM)

El módulo `uncertainty.py` implementa propagación GUM simplificada. La incertidumbre base de desplazamiento en píxeles σ_{px} se estima a partir del error de reproyección de la calibración (si disponible) o se asigna un valor conservador de 0.05 px para DIC 2D subpíxel:

$$\sigma_{\text{disp}}(i, k) = \sigma_{\text{px}} \cdot p(C_{\text{ZNCC}}(i, k)), \quad (7)$$

donde $p(C)$ es un factor de penalización que aumenta la incertidumbre cuando el coeficiente de correlación se aleja de la unidad:

$$p(C) = 1 + \text{clip}\left(\frac{1-C}{0.10}, 0, 5\right). \quad (8)$$

La incertidumbre combinada del vector de desplazamiento es:

$$u_c = \sqrt{\sigma_x^2 + \sigma_y^2}. \quad (9)$$

La propagación a strain considera que cada componente es esencialmente una derivada numérica sobre la longitud efectiva del gauge virtual $L_x = 2r \Delta_x$ y $L_y = 2r \Delta_y$:

$$u(\varepsilon_{xx}) = \frac{\sqrt{2} \sigma_u}{L_x}, \quad u(\varepsilon_{yy}) = \frac{\sqrt{2} \sigma_v}{L_y}, \quad u(\varepsilon_{xy}) = \frac{1}{2} \sqrt{\left(\frac{\sqrt{2} \sigma_u}{L_y}\right)^2 + \left(\frac{\sqrt{2} \sigma_v}{L_x}\right)^2}. \quad (10)$$

3 Estructura del paquete

El paquete distribuido en `DIC_2D_V3_1_mejorado.zip` contiene los archivos mostrados en la Tabla 1.

Archivo	Responsabilidad en el pipeline
<code>run_dic_V3.py</code>	Orquestador principal. Define todos los parámetros y ejecuta la cadena completa de análisis.
<code>DIC_2D_V3/config.py</code>	Clases de configuración: <code>DICConfigV3</code> y subclases por responsabilidad (<code>ComputeConfig</code> , <code>ImagingConfig</code> , <code>ROIConfig</code> , etc.).
<code>DIC_2D_V3/project.py</code>	Clase <code>DICProjectV3</code> : contenedor del estado del proyecto y de todas las matrices de resultados.
<code>DIC_2D_V3/image_loader.py</code>	Carga de imágenes, validación de forma común y normalización a <code>float32</code> en escala de grises.
<code>DIC_2D_V3/calibration.py</code>	Metrología de escala mm/px (interactiva o desde archivo), corrección de distorsión óptica y persistencia de calibración.
<code>DIC_2D_V3/roi_grid.py</code>	Definición de ROI rectangular interactiva o programática; generación de la grilla reducida de puntos de análisis.
<code>DIC_2D_V3/correlation_engine.py</code>	Motor de correlación ZNCC con tracking incremental, predictor local, refinamiento subpíxel parabólico y fallback absoluto.
<code>DIC_2D_V3/displacement.py</code>	Filtrado de outliers, relleno de huecos, suavizado espacial y construcción de campos de desplazamiento en px y mm.
<code>DIC_2D_V3/strain.py</code>	Gradientes de desplazamiento por LSQ local (displacement tipo NCorr) y cálculo de tensores de deformación.
<code>DIC_2D_V3/uncertainty.py</code>	Estimación de incertidumbre GUM para desplazamiento y deformación con propagación analítica.
<code>DIC_2D_V3/visualization.py</code>	Dashboard de resultados, campo vectorial (quiver), mapas de campos escalares, curvas temporales y tabla GUM.
<code>DIC_2D_V3/utils.py</code>	Temporizador, configuración de logging y funciones auxiliares compartidas.

Cuadro 1: Estructura funcional del paquete `DIC_2D_V3`.

Importante

El archivo de inicialización del paquete está nombrado como `DIC_2D_V3___init__.py` en el ZIP distribuido. Para que Python lo reconozca correctamente como `__init__.py` del paquete, debe renombrarse a `__init__.py` dentro del directorio `DIC_2D_V3/` antes de ejecutar el pipeline.

4 Parámetros de entrada del script principal

Todos los parámetros de configuración se encuentran en los diccionarios al inicio de `run_dic_v3.py`, en la sección marcada con el comentario `# PARÁMETROS DE ENTRADA - EDITAR AQUÍ`. El usuario debe editar únicamente esa sección. Los módulos del paquete `DIC_2D_V3/` no deben modificarse para operación normal.

4.1 Bloque PROJECT: rutas y nombre del proyecto

Parámetro	Valor predeterminado	Descripción
<code>name</code>	<code>"pw-0mil-p2-n14"</code>	Identificador del proyecto. Se usa para nombrar el archivo de estado <code>.dicv3</code> y los archivos de calibración.
<code>image_dir</code>	<code>.\imagenes</code>	Directorio que contiene la secuencia de imágenes del ensayo.
<code>image_pattern</code>	<code>"pw-0mil-p2-n14*.tif"</code>	Patrón glob para seleccionar los archivos de imagen. El orden de carga sigue el orden alfanumérico de los nombres.
<code>output_dir</code>	<code>.\resultados_dic_v3</code>	Directorio de destino para todas las salidas: figuras, archivos de datos y estado del proyecto.

4.2 Bloque COMPUTE: cómputo y aceleración

Parámetro	Valor predeterminado	Descripción
<code>use_gpu</code>	<code>True</code>	Activa la detección de GPU vía PyTorch.
<code>gpu_backend</code>	<code>"torch"</code>	Backend de aceleración. Opciones: <code>"torch"</code> , <code>"none"</code> .
<code>gpu_device</code>	<code>"auto"</code>	Selección de dispositivo: <code>"auto"</code> elige CUDA si está disponible, <code>"cpu"</code> fuerza CPU.
<code>gpu_for_correlation</code>	<code>False</code>	La correlación ejecuta en CPU en esta versión. La GPU no participa en el núcleo de matching.
<code>gpu_for_visualization</code>	<code>True</code>	El remuestreo de campos para visualización puede aprovechar GPU mediante PyTorch.
<code>num_threads</code>	<code>0</code>	Número de hilos de OpenCV/NumPy. El valor 0 selecciona automáticamente el número óptimo.

Nota técnica

La correlación DIC en esta versión (V3.1) ejecuta íntegramente en CPU con operaciones de OpenCV y NumPy. El indicador `gpu_for_correlation = False` es correcto y no debe modificarse a `True` en esta versión, ya que la lógica de tracking incremental no tiene implementación GPU.

4.3 Bloque IMAGING: imagen y calibración

Parámetro	Valor predeterminado	Descripción
<code>apply_calibration</code>	True	Activa la etapa de calibración/metrología. Si es False, todos los resultados quedan en píxeles.
<code>interactive_calibration</code>	True	Solicita al usuario seleccionar dos puntos sobre la imagen de referencia y teclear la distancia real en mm.
<code>use_saved_calibration_if_available</code>	True	Si existe un archivo de calibración previo (.npz o .json), lo reutiliza sin solicitar interacción.
<code>distortion_correction</code>	True	Aplica corrección de distorsión radial si el archivo de calibración incluye los parámetros intrínsecos de la cámara (<code>camera_matrix</code> , <code>dist_coeffs</code>).
<code>output_units</code>	"mm"	Unidades de salida para los campos de desplazamiento: "mm" requiere calibración válida; "px" no.
<code>strain_display_units</code>	"microstrain"	Escala de visualización de la deformación. El cálculo interno es siempre adimensional; la conversión a $\mu\epsilon$ se aplica solo en figuras y archivos de exportación secundarios.

4.4 Bloque ROI: región de interés y grilla de análisis

Parámetro	Valor predeterminado	Descripción
interactive	True	Si es True, la ROI se selecciona manualmente sobre la imagen de referencia haciendo clic en dos esquinas opuestas.
type	"rectangular"	Tipo de ROI. En esta versión, la opción activa es "rectangular". El módulo soporta ROI poligonal programática, pero no es el flujo predeterminado.
spacing_x	15	Separación horizontal entre centros de subset, en píxeles. Determina la densidad de la grilla de análisis en x .
spacing_y	15	Separación vertical entre centros de subset, en píxeles. Determina la densidad de la grilla de análisis en y .

Vínculo con Manual del Participante

La relación entre el espaciado de la grilla Δ , el radio del subset r y la resolución espacial del strain se analiza en detalle en el §4.3.2 y §4.3.3 del Manual del Participante. El *Virtual Strain Gauge* efectivo tiene longitud $L = 2r_s \Delta$, donde r_s es el `strain_radius`.

4.5 Bloque DIC: parámetros del motor de correlación

Parámetro	Valor predeterminado	Descripción
subset_radius	21	Radio del subset en píxeles. El subset tiene dimensión $(2r + 1) \times (2r + 1) = 43 \times 43$ px. Subsets más grandes aumentan la resistencia al ruido pero reducen la resolución espacial.
subset_spacing	15	Paso de la grilla. Asignado a <code>cfg.dic.spacing</code> . Debe coincidir con <code>ROI["spacing_x"]</code> y <code>ROI["spacing_y"]</code> para consistencia.
cutoff_diffnorm	1e-6	Criterio de convergencia para el refinamiento iterativo (no activo en el motor NCC; reservado para implementaciones IC-GN futuras).
cutoff_iteration	50	Número máximo de iteraciones permitidas (idem anterior).
step_analysis	True	Activa la correlación incremental frame a frame. Ver §2.4.
corrcoef_min	0.50	Umbral mínimo del coeficiente ZNCC para aceptar un punto como válido. Valores por debajo indican correlación poco confiable.
search_radius	23	Radio de la ventana de búsqueda alrededor de la posición predicha. Debe ser mayor que el desplazamiento máximo entre frames consecutivos.
mode	"incremental"	Modo de correlación: "incremental" = frame a frame acumulado; "absolute" = todos los frames contra la referencia.
seed_strategy	"propagation"	Estrategia de propagación de semillas para el inicio de la correlación.
use_predictor	True	Activa el predictor basado en el incremento del frame anterior. Mejora la tasa de éxito en ensayos con grandes deformaciones.
engine	"rgdic"	Motor declarado. En V3.1 el matching NCC es el kernel efectivo.
interpolation	"bicubic"	Interpolación para extracción sub-píxel del template: "bicubic" → <code>cv2.INTER_CUBIC</code> , "bilinear" → <code>cv2.INTER_LINEAR</code> .
warp	"affine"	Modelo de deformación local del subset. En V3.1 se usa traslación pura; el campo "affine" es declarativo.

Recomendación operativa

Para elegir `subset_radius`, el criterio práctico es que el subset debe contener al menos 3–5 manchas del patrón moteado. Si el speckle tiene tamaño típico de 5–8 px, un radio de 15–25 px es adecuado. Un radio excesivo produce suavizado espacial; uno insuficiente produce correlaciones ruidosas.

4.6 Bloque DISPLACEMENT: postproceso de desplazamiento

Parámetro	Valor predeterminado	Descripción
<code>outlier_method</code>	"corrcoef+median"	Método de detección de puntos anómalos. Combina umbralado por coeficiente de correlación y filtro de mediana local sobre los desplazamientos.
<code>median_threshold</code>	2.5	Umbral en desviaciones estándar robustas para detección de outliers por mediana. Puntos que se alejan más de este umbral son descartados.
<code>spatial_smoothing</code>	"gaussian"	Suavizado espacial de los campos de desplazamiento postprocesados. Opciones: "none", "gaussian", "median".
<code>smoothing_kernel</code>	3	Tamaño del kernel de suavizado en unidades de nodos de grilla.
<code>fill_missing</code>	True	Rellena los nodos inválidos por interpolación sobre los vecinos válidos.
<code>fill_method</code>	"griddata"	Método de interpolación espacial para relleno de huecos. "griddata" usa interpolación triangular irregular mediante Delaunay.
<code>export_displacements_in_mm</code>	True	Genera versiones métricas (<code>disp_x_mm</code> , <code>disp_y_mm</code> , <code>disp_mag_mm</code>) cuando <code>mm_per_px</code> es válido.

4.7 Bloque STRAIN: deformación

Parámetro	Valor predeterminado	Descripción
engine	"ncorr_dispgrad"	Algoritmo de cálculo de gradientes de desplazamiento. El método activo aproxima el dispgrad de NCorr: ajuste LSQ lineal local sobre vecindad circular. Alternativa: "gradient" (diferencias finitas por <code>numpy.gradient</code>).
strain_radius	7	Radio de la vecindad circular sobre la grilla reducida para el ajuste LSQ. Define el tamaño del VSG efectivo: $L = 2 \times 7 \times 15 = 210 \text{ px} \approx$ con la escala del ensayo.
tensor	"infinitesimal"	Tipo de tensor de deformación. Opciones: "infinitesimal" (pequeñas deformaciones), "green_lagrange" (grandes deformaciones), "eulerian_almansi".
compute_shear	True	Calcula y exporta la componente constante ε_{xy} .
compute_principal	True	Calcula deformaciones principales $\varepsilon_1, \varepsilon_2$ y ángulo θ .
exclude_boundary	True	Excluye los nodos del borde de la grilla del cálculo de strain, donde la vecindad circular queda incompleta.

4.8 Bloque UNCERTAINTY: incertidumbre

Parámetro	Valor predeterminado	Descripción
enabled	True	Activa el módulo de estimación de incertidumbre.
mode	"gum"	Modo de propagación. El modo "gum" aplica las fórmulas analíticas del §2.6. El modo "none" omite el cálculo.

4.9 Bloque VIS: visualización y exportación

Parámetro	Valor predeterminado	Descripción
dpi	180	Resolución de las figuras exportadas en puntos por pulgada.
export_format	"png"	Formato de exportación de figuras. Opciones estándar de Matplotlib: "png", "pdf", "svg".
smooth_method	"bspline"	Método de suavizado para remuestreo visual de campos escalares sobre una grilla regular de alta resolución.
upsample_nx, upsample_ny	320	Resolución de la grilla interpolada para visualización del dashboard y mapas de campo.
levels	140	Número de niveles de color para mapas de contorno.
quiver_skip	6	Factor de submuestreo del campo vectorial. Se muestra 1 de cada quiver_skip flechas en cada dirección.
quiver_gain	3.0	Factor de escala de las flechas del quiver.

4.10 Bloque RUNTIME: control de ejecución

Parámetro	Valor predeterminado	Descripción
<code>selected_frame</code>	40	Índice del frame DIC (base 0) para el que se generan figuras de inspección detallada al final del pipeline. El índice de imagen asociado es <code>selected_frame + 1</code> .
<code>min_valid_points_per_frame</code>	300	Número mínimo de nodos con correlación exitosa requeridos para que un frame sea considerado usable. Frames con menos puntos son descartados automáticamente.
<code>trim_correlation_tail</code>	True	Activa el recorte automático de los frames finales en los que la correlación colapsa por pérdida de patrón o desplazamiento excesivo.
<code>show_selected_frame_results</code>	True	Genera el conjunto completo de figuras de inspección para el frame seleccionado.

5 Descripción técnica de los módulos

5.1 Carga de imágenes: `image_loader.py`

La función `run_image_loader_v3()` realiza las siguientes operaciones:

1. Búsqueda de archivos en `image_dir` que coinciden con `image_pattern`, usando `glob` con ordenamiento alfanumérico.
2. Carga de cada imagen con OpenCV o PIL y conversión a escala de grises (float32, rango `[0, 1]`).
3. Verificación de que todas las imágenes tienen la misma resolución. Si hay inconsistencias, se lanza una excepción.
4. Almacenamiento de la lista de imágenes en `project.images` y de los metadatos de la secuencia en `project.image_shape`, `project.num_images`.
5. Generación de una vista previa con la primera, la intermedia y la última imagen de la secuencia (Figura 1).

La imagen en el índice `reference_index = 0` se usa como referencia para todo el análisis.

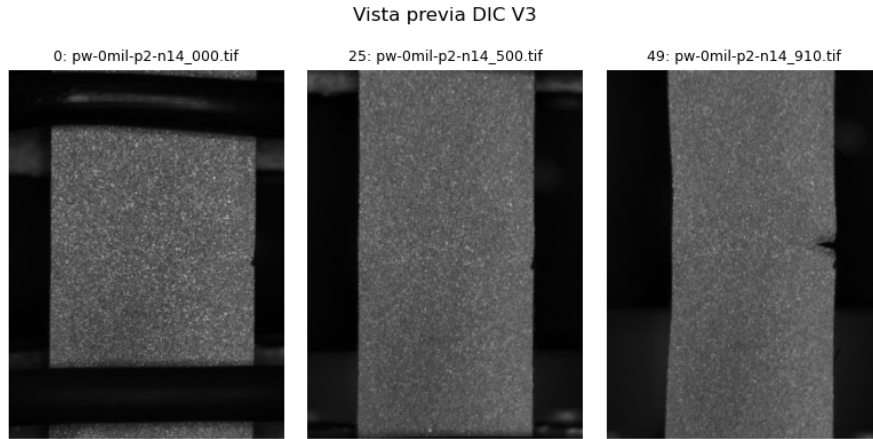


Figura 1: Vista previa de la secuencia experimental generada por `image_loader.py`. Permite verificar consistencia de iluminación, resolución y presencia de deformación observable antes de correlacionar.

5.2 Calibración: `calibration.py`

La función `run_calibration_v3()` ejecuta tres operaciones en secuencia:

1. **Carga de calibración guardada.** Busca archivos `.npz` o `.json` con el nombre `<project_name>_calib` en el directorio de salida y en el directorio de imágenes. Si encuentra uno válido y `use_saved = True`, lo carga sin requerir interacción del usuario.
2. **Calibración interactiva de escala.** Si no existe calibración previa y `interactive = True`, se muestra la imagen de referencia y el usuario selecciona dos puntos con `ginput`. Tras ingresar la distancia real en mm en la consola, el sistema calcula:

$$\text{mm/px} = \frac{d_{\text{real}} [\text{mm}]}{d_{\text{px}} [\text{px}]}, \quad (11)$$

donde $d_{\text{px}} = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$.

3. **Corrección de distorsión.** Si el archivo de calibración contiene `camera_matrix` (matriz intrínseca 3×3) y `dist_coeffs` (coeficientes de distorsión radial/tangencial), se aplica `cv2.undistort()` a toda la secuencia de imágenes.

La calibración se guarda automáticamente al final del paso en formatos `.npz` y `.json` para reutilización en corridas posteriores.

Vínculo con Manual del Participante

El proceso de calibración de cámara con tablero de ajedrez (Zhang, 2000), el factor de escala y su propagación de incertidumbre se describen en el Capítulo 3 del Manual del Participante.

5.3 ROI y grilla: `roi_grid.py`

La función `run_grid_definition_v3()` opera de la siguiente manera:

En modo interactivo (`interactive = True`), muestra la imagen de referencia y solicita al usuario que defina dos esquinas opuestas del rectángulo de análisis. El módulo aplica automáticamente un margen interno de `subset_radius` píxeles para garantizar que los subsets de los nodos del borde no excedan los límites de la imagen.

Sobre el rectángulo así definido, se genera una grilla ortogonal regular con el paso (`spacing_x`, `spacing_y`). Las coordenadas de los N nodos se almacenan en `project.grid_x` y `project.grid_y` como vectores de longitud N .

La figura resultante de selección de ROI se muestra en la consola y puede verse en la Figura 2.

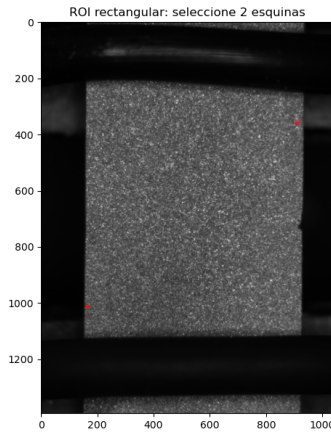


Figura 2: Selección de ROI rectangular interactiva. El contorno verde delimita la región aceptada. Los nodos de la grilla se ubican dentro de ese borde, respetando el margen de `subset_radius`.

5.4 Motor de correlación: `correlation_engine.py`

La función `run_correlation_v3()` implementa el siguiente ciclo para cada frame $k = 1, \dots, N_{\text{frames}}$ y cada punto de la grilla $i = 1, \dots, N_{\text{pts}}$:

1. **Selección de imágenes.** En modo incremental: $I_{\text{prev}} = I_k$, $I_{\text{curr}} = I_{k+1}$. En modo absoluto: $I_{\text{prev}} = I_0$, $I_{\text{curr}} = I_{k+1}$.

2. **Extracción del template.** El template de dimensión $(2r + 1)^2$ se extrae de I_{prev} centrado en la posición material actual del punto: $(x_0 + u_{k-1}, y_0 + v_{k-1})$.
3. **Predicción.** La ventana de búsqueda se centra en el predictor: $(x_0 + u_{k-1} + \Delta u_{k-1}, y_0 + v_{k-1} + \Delta v_{k-1})$, donde $\Delta u_{k-1}, \Delta v_{k-1}$ son los incrementos del frame previo.
4. **Matching ZNCC.** Se evalúa `cv2.matchTemplate(TM_CCOEFF_NORMED)` sobre la ventana de búsqueda y se localiza el máximo del mapa de correlación.
5. **Refinamiento subpíxel.** Se aplica interpolación parabólica 1D al pico del mapa de correlación (función `_parabolic_subpixel_1d`).
6. **Aceptación o fallback.** Si $C_{\text{ZNCC}} \geq \text{corrcoef_min}$, el punto es aceptado (`flag = 1`). En caso contrario, se intenta un fallback de correlación absoluta usando el template de la imagen de referencia global (I_0) y una ventana de búsqueda ampliada. Si el fallback también falla, el punto queda inválido (`flag = 0`).
7. **Acumulación.** Los desplazamientos acumulados se actualizan: $u_k = u_{k-1} + \Delta u_k, v_k = v_{k-1} + \Delta v_k$.

Las matrices resultado son de dimensión $N_{\text{pts}} \times N_{\text{frames}}$: `project.corr_u`, `project.corr_v` (desplazamientos en px), `project.corr_coeff` (coeficiente ZNCC), `project.corr_flags` (0 = inválido, 1 = válido).

V3 Corr 9/49 | cc=0.9315 | ok=2050/2050

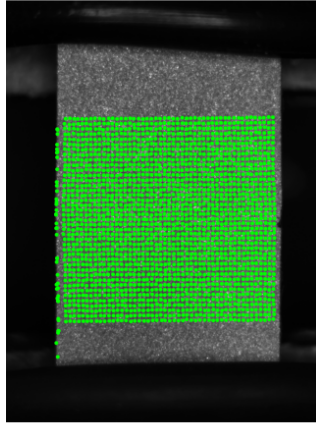


Figura 3: Control visual de correlación sobre un frame. Los nodos verdes representan puntos con $C_{\text{ZNCC}} \geq \text{corrcoef_min}$; los rojos son puntos rechazados o para los que el fallback también falló.

5.4.1 Recorte de cola de correlación

Después de la correlación, la función `_summarize_and_trim_correlation()` descarta los frames finales en que el número de puntos válidos cae por debajo de `min_valid_points_per_frame`. Este mecanismo protege el cálculo de strain de frames con densidad insuficiente de puntos, que degeneran la estimación de gradientes.

5.5 Postproceso de desplazamiento: `displacement.py`

La función `run_displacement_v3()` aplica en secuencia:

1. Detección de outliers por combinación de umbral en `corr_coeff` y filtro de mediana sobre los desplazamientos con umbral `median_threshold`.
2. Relleno de nodos inválidos por interpolación espacial (`fill_method`).
3. Suavizado espacial gaussiano con kernel `smoothing_kernel`.
4. Construcción de los campos exportables: `project.disp_x_px`, `project.disp_y_px`, y si existe calibración válida, `project.disp_x_mm`, `project.disp_y_mm` y sus magnitudes.

Antes de calcular el strain, la función `_repair_bad_frames_temporal()` identifica frames con ratio de validez < 0.50 y reemplaza sus desplazamientos por interpolación lineal temporal entre los frames buenos vecinos. Esto previene que frames espurios contaminen la derivación numérica del strain.

5.6 Cálculo de strain: `strain.py`

La función `run_strain_v3()` implementa el flujo descrito en §2.5. Las consideraciones de implementación relevantes son:

- Si existen desplazamientos en mm (`disp_x_mm`, `disp_y_mm`) y `mm_per_px` es válido, los gradientes se calculan en unidades mm/mm (strain adimensional). De lo contrario, se trabaja en px/px, lo que da el mismo resultado adimensional si la grilla es uniforme.
- El parámetro `exclude_boundary = True` marca como NaN los nodos dentro de la banda de `strain_radius` nodos del borde de la grilla, donde la vecindad circular quedaría incompleta.
- Las deformaciones principales ε_1 , ε_2 y el ángulo θ se calculan analíticamente a partir de las componentes del tensor mediante las fórmulas del círculo de Mohr.

5.7 Incertidumbre: `uncertainty.py`

La función `run_uncertainty_v3()` implementa el modo "gum" como se describe en §2.6. Los resultados almacenados en el proyecto son:

- `project.ux`, `project.uy`: incertidumbre estándar de los desplazamientos $u(U)$ y $u(V)$, en las mismas unidades que los desplazamientos (mm o px).
- `project.u_combined`: incertidumbre combinada del vector de desplazamiento.
- `project.ustrain_xx`, `project.ustrain_yy`, `project.ustrain_xy`: incertidumbre de las componentes del tensor de deformación, adimensionales.

6 Ejecución del pipeline

6.1 Prerrequisitos del sistema

- Python ≥ 3.9
- Bibliotecas: `numpy`, `scipy`, `opencv-python`, `matplotlib`, `pathlib` (estándar).
- Opcional: `torch` ≥ 1.10 para aceleración GPU en visualización.
- Backend gráfico: `Qt5Agg` (configurado en `run_dic_V3.py`). Se requiere `PyQt5` o `PySide2`.

Importante

El script configura el backend de Matplotlib con `matplotlib.use("Qt5Agg")` antes del primer `import matplotlib.pyplot`. Esto debe ocurrir antes de cualquier otra importación de Matplotlib. Si el entorno no dispone de Qt5, cambie esta línea a `"TkAgg"` o el backend disponible en su instalación.

6.2 Estructura de directorios esperada

Listing 1: Estructura mínima antes de ejecutar el pipeline.

```
1      proyecto/  
2      run_dic_V3.py  
3      DIC_2D_V3/  
4      __init__.py          # renombrado desde DIC_2D_V3__init__.py  
5      calibration.py  
6      config.py  
7      correlation_engine.py  
8      displacement.py  
9      image_loader.py  
10     project.py  
11     roi_grid.py  
12     strain.py  
13     uncertainty.py  
14     utils.py  
15     visualization.py  
16     imagenes/  
17     pw-0mil-p2-n14_001.tif  
18     pw-0mil-p2-n14_002.tif  
19     ...
```

6.3 Ejecución estándar

Listing 2: Ejecución del pipeline desde la raíz del proyecto.

```
1 python run_dic_v3.py
```

El flujo efectivo dentro de `main()` sigue la secuencia:

1. Creación del proyecto `DICProjectV3` con la configuración construida por `build_config()`.
2. Carga de imágenes con `run_image_loader_v3()`.
3. Calibración de escala y corrección óptica con `run_calibration_v3()`.
4. Definición de ROI y generación de grilla con `run_grid_definition_v3()`.
5. Correlación incremental con `run_correlation_v3()`.
6. Resumen de correlación y recorte de cola con `_summarize_and_trim_correlation()`.
7. Postproceso de desplazamientos con `run_displacement_v3()`.
8. Reparación temporal de frames anómalos con `_repair_bad_frames_temporal()`.
9. Cálculo de strain con `run_strain_v3()`.
10. Estimación de incertidumbre con `run_uncertainty_v3()`.
11. Generación de figuras globales (último frame válido).
12. Generación de figuras del frame seleccionado (`selected_frame = 40`).
13. Guardado del proyecto con `proj.save()`.

7 Interpretación de las salidas

7.1 Dashboard global

El dashboard se genera con `plot_dashboard_v3()` para el último frame válido. Presenta seis paneles en dos filas (Figura 4):

- Fila superior: desplazamientos U (dirección x), V (dirección y) y magnitud $|d| = \sqrt{U^2 + V^2}$, en mm si la calibración es válida.
- Fila inferior: deformaciones ε_{xx} , ε_{yy} y ε_{xy} , escaladas en $\mu\epsilon$ si `strain_display_units = "microstrain"`.

Los campos se remuestrean sobre una grilla regular de 320×320 puntos mediante interpolación B-spline para mejorar la apariencia visual, pero los valores de color corresponden a los datos calculados.

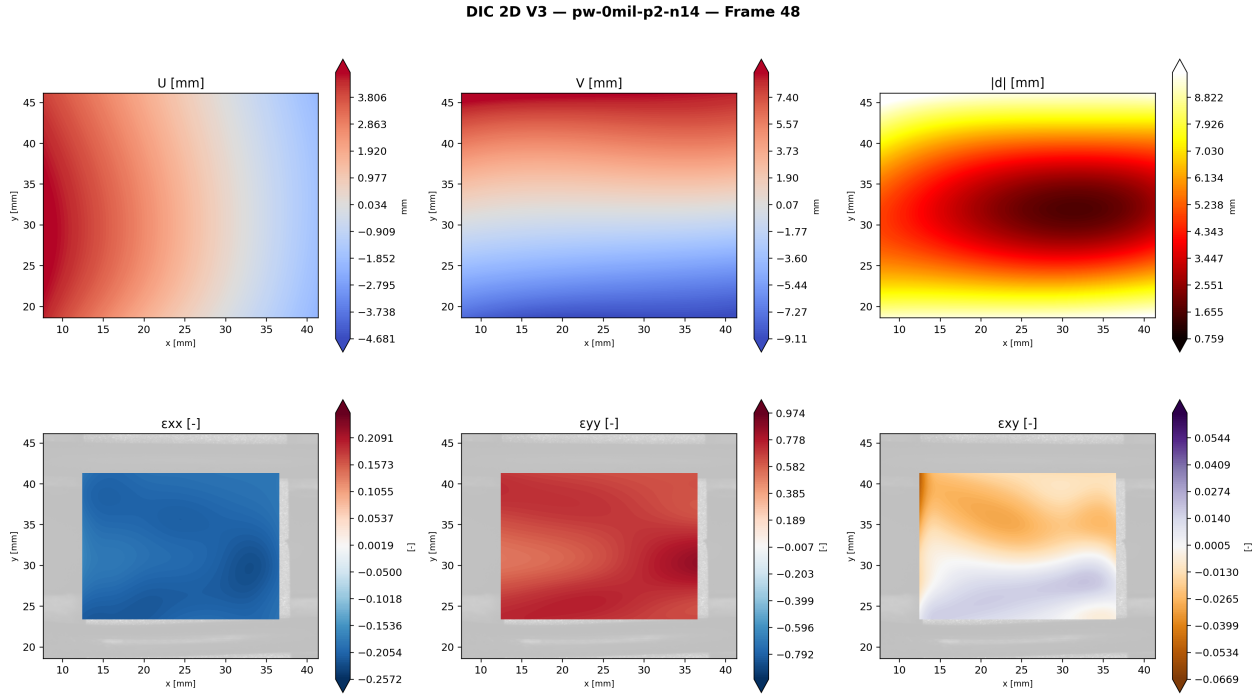


Figura 4: Dashboard global del último frame válido. Fila superior: campos de desplazamiento U , V y magnitud $|d|$. Fila inferior: componentes del tensor de deformación ε_{xx} , ε_{yy} y ε_{xy} .

7.2 Campo vectorial de desplazamiento (quiver)

La figura `quiver.png` muestra el campo de desplazamiento como vectores (Figura 5). Cada flecha representa el vector (U, V) en un nodo de la grilla submuestreado por `quiver_skip`. El color codifica la magnitud $|d|$ y la longitud de las flechas está escalada por `quiver_gain`. Esta figura permite verificar la coherencia espacial del campo cinemático y detectar discontinuidades espurias o artefactos de correlación.

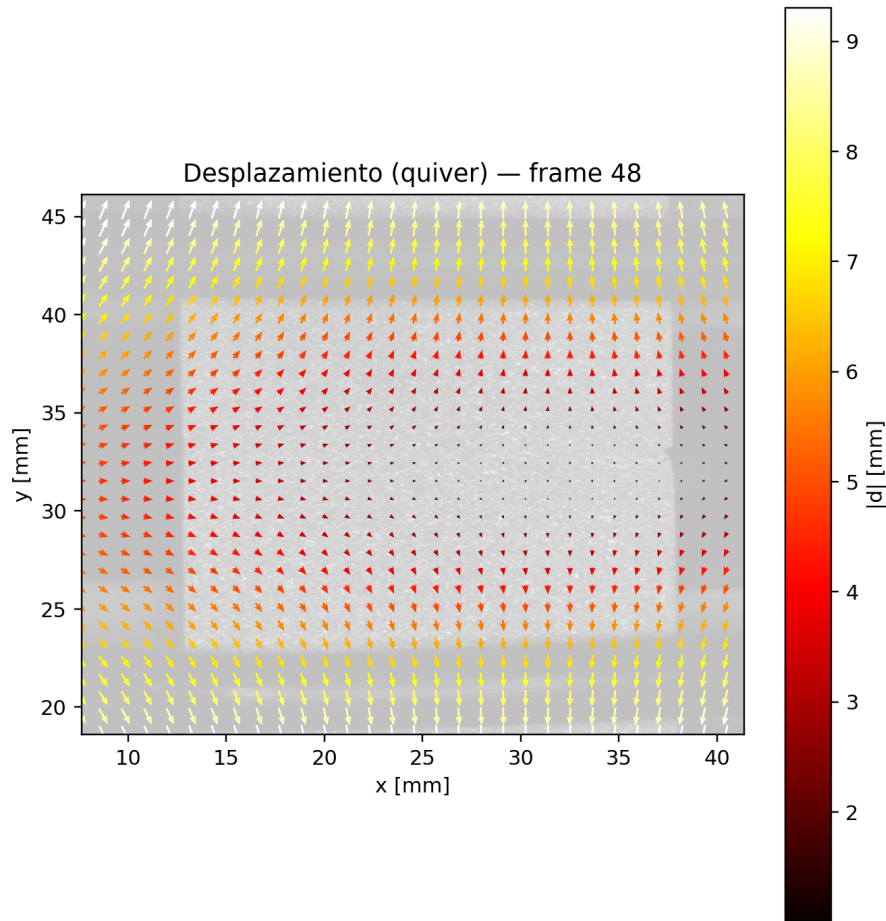


Figura 5: Campo vectorial de desplazamiento. La dirección de cada flecha indica el sentido del desplazamiento; el color codifica la magnitud $|d|$.

7.3 Mapas individuales de deformación

Los archivos `strain_xx.png`, `strain_yy.png` y `strain_xy.png` muestran cada componente del tensor por separado (Figura 6). La escala de color es simétrica respecto a cero y se ajusta automáticamente a los percentiles [2, 98] para evitar que valores atípicos aislados saturen la escala cromática.

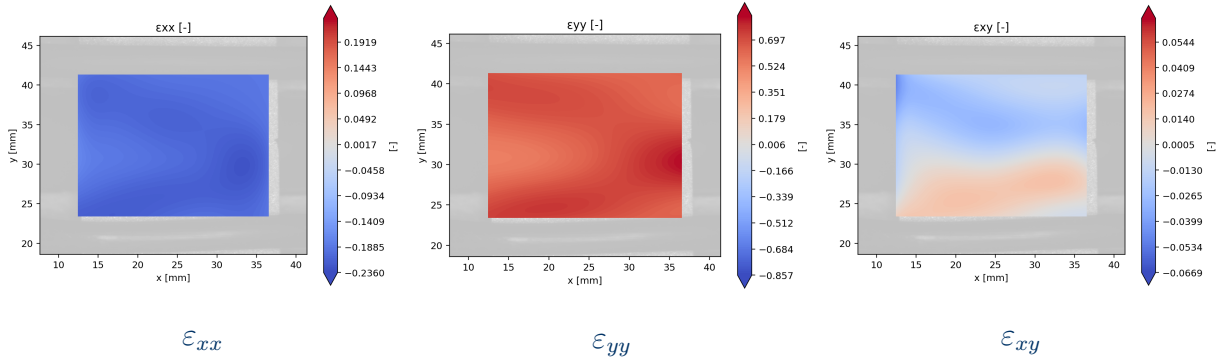


Figura 6: Mapas individuales de deformación. ε_{xx} : deformación normal en x ; ε_{yy} : deformación normal en y ; ε_{xy} : componente cortante. En un ensayo de tracción uniaxial, ε_{xx} debe dominar y ε_{yy} mostrar contracción por efecto de Poisson.

7.4 Evolución temporal de la deformación

La figura `strain_vs_frame.png` (Figura 7) muestra la media espacial de cada componente del tensor en función del índice de frame. Esta curva permite:

- Identificar etapas de carga lineal, no lineal o descarga.
- Detectar frames con correlación degradada (saltos abruptos que no corresponden a cambios físicos reales).
- Verificar que el recorte de cola de correlación se activó en el momento correcto.

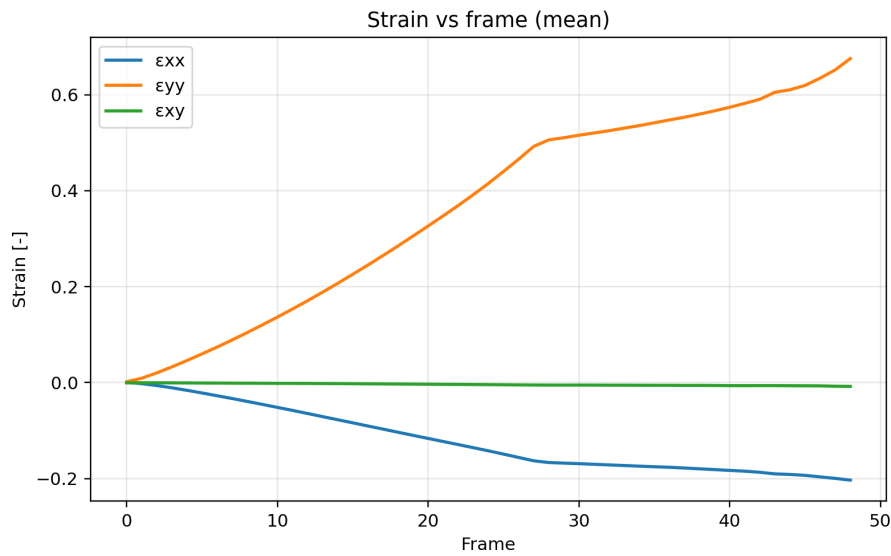


Figura 7: Evolución temporal de la media espacial de ε_{xx} , ε_{yy} y ε_{xy} en función del frame DIC.

7.5 Strain vs. desplazamiento máximo

La figura `strain_vs_dmax.png` (Figura 8) relaciona la media espacial de ε_{xx} y ε_{yy} con el desplazamiento máximo $|d|_{\max}$ del frame correspondiente. Esta representación es útil para detectar no linealidades globales entre la cinemática y la deformación, o para verificar que la relación carga–deformación es consistente con el material y el ensayo.

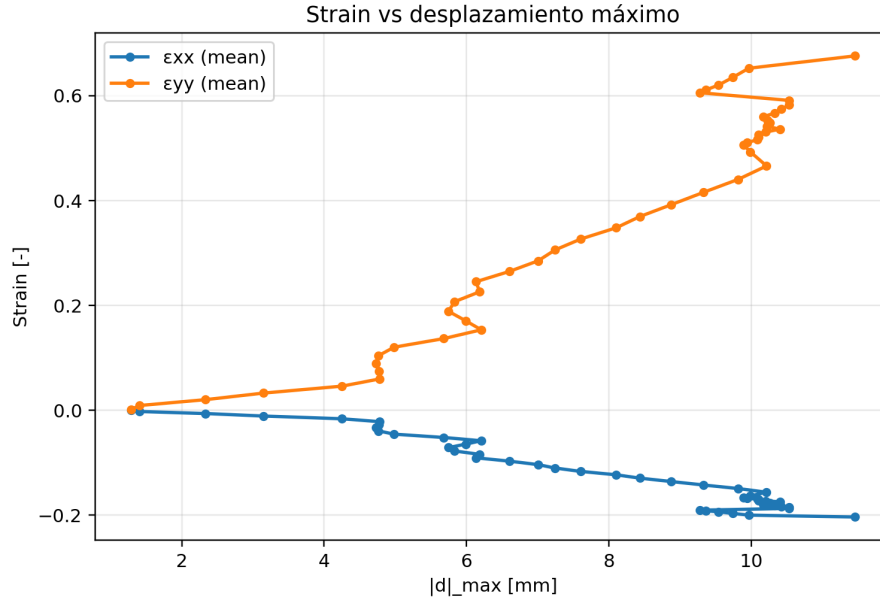


Figura 8: Relación entre la deformación media ε_{xx} , ε_{yy} y el desplazamiento máximo $|d|_{\max}$ por frame.

7.6 Resumen de incertidumbre

La figura `uncertainty_summary.png` (Figura 9) presenta el mapa espacial de la incertidumbre estándar de los desplazamientos $u(U)$, $u(V)$, la incertidumbre combinada u_c y la incertidumbre del cortante $u(\varepsilon_{xy})$ para el último frame disponible.

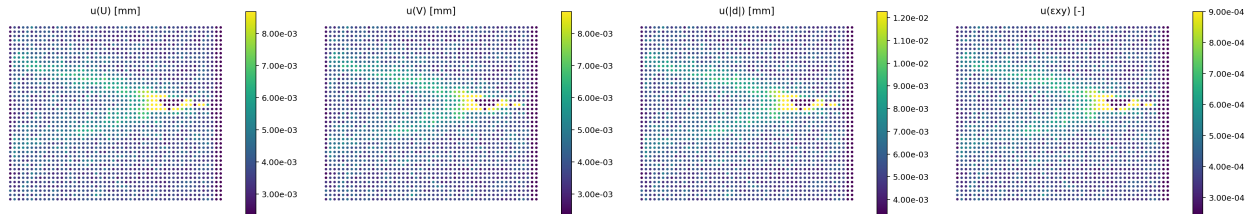


Figura 9: Resumen espacial de incertidumbre GUM. Fila superior: $u(U)$, $u(V)$ y u_c en las unidades de desplazamiento activas. Fila inferior: $u(\varepsilon_{xy})$ adimensional.

La incertidumbre de desplazamiento no es uniforme: los nodos con coeficiente ZNCC bajo reciben un factor de penalización que incrementa su incertidumbre estimada. Zonas con patrón moteado degradado o bordes del ensayo mostrarán valores más altos.

7.7 Tabla GUM

La figura `gum_table.png` (Figura 10) presenta el presupuesto de incertidumbre en formato tabular: fuente de incertidumbre, valor numérico y contribución relativa a la incertidumbre combinada.

Presupuesto de incertidumbre GUM — pw-0mil-p2-n14

Incertidumbre de desplazamiento

Fuente de error	Tipo	Distribución	u_i (base)	c_i (sensib.)	Contribución $ c_i \cdot u_i $
Interpolación subpixel	B	Rectangular	0.0500 px	0.045911	2.2956e-03 mm
Ruido de imagen	A	Normal	-0.0150 px	0.045911	6.8867e-04 mm
Calidad de correlación	A	Normal	cc-penalty $\times \sigma$	variable	incl. en $u(U)$, $u(V)$
Calibración mm/px	B	Rectangular	2.2956e-04 mm/px	$ dj $	-2.2956e-04 mm
$u_c(U)$					4.0189e-03 mm
$u_c(V)$					4.0189e-03 mm
$u_c(d)$					5.6835e-03 mm

Propagación a strain (infinitesimal)

Fuente de error	Tipo	Distribución	u_i (base)	c_i (sensib.)	Contribución $ c_i \cdot u_i $
$u(U)$, $u(V)$ — gradiente	B	Propagación	$\sqrt{2} \cdot \sigma_u / L$	$1/L_x = 0.1037$	5.8950e-04
Strain radius $r=7$			$L_x = 2r \cdot \Delta x = 9.6414$ mm	$L_y = 9.6414$	
Grid spacing $\Delta x = 0.69$, $\Delta y = 0.69$ mm					
$u_c(ε_{xx})$					5.8950e-04
$u_c(ε_{yy})$					5.8950e-04
$u_c(ε_{xy})$					4.1684e-04

Figura 10: Tabla de presupuesto de incertidumbre GUM generada por `plot_gum_table_v3()`.

8 Salidas del frame seleccionado

Al finalizar el pipeline global, el bloque `_show_selected_frame_results()` genera un conjunto de figuras específicas para el frame DIC con índice `selected_frame = 40`. El índice de imagen correspondiente es `40 + 1 = 41`.

Las figuras generadas para ese frame son las mismas que en el análisis global (dashboard, quiver, mapas de ε_{xx} , ε_{yy} , ε_{xy}), con los sufijos `_frame_040` en los nombres de archivo. Adicionalmente, se muestra la imagen original asociada al frame y el mapa de validez de correlación para ese instante.

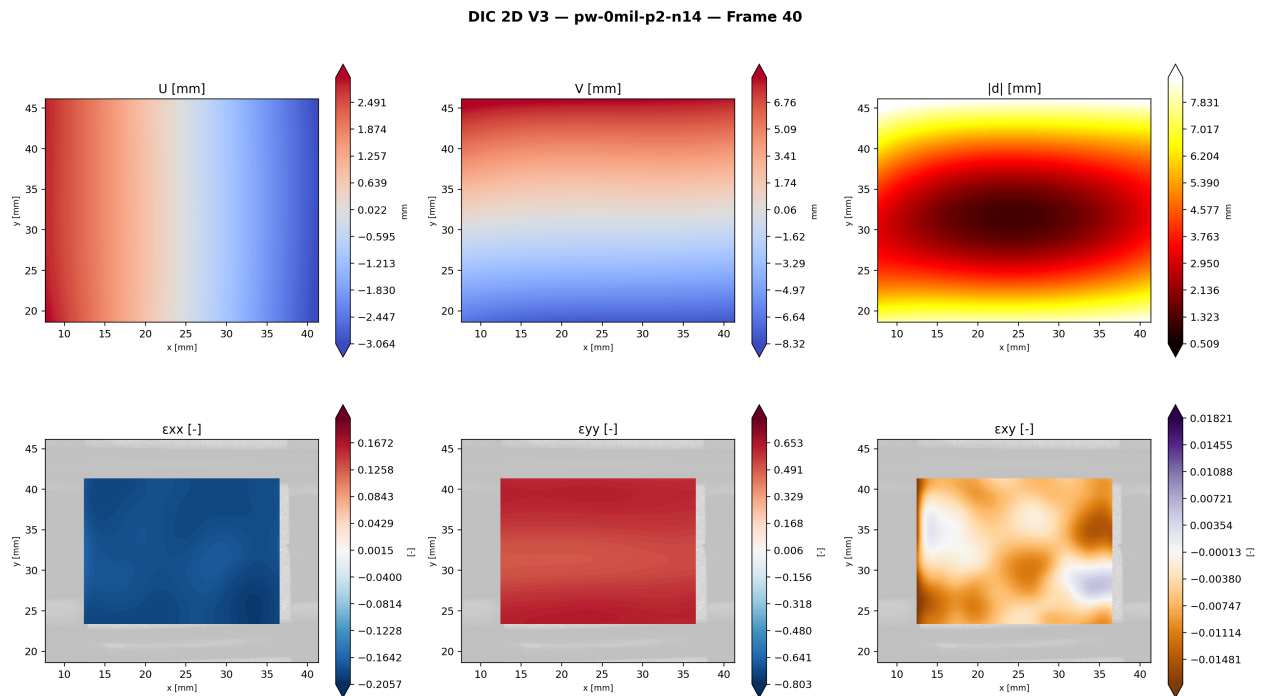


Figura 11: Dashboard de inspección para el frame seleccionado (frame DIC 40, imagen 41).

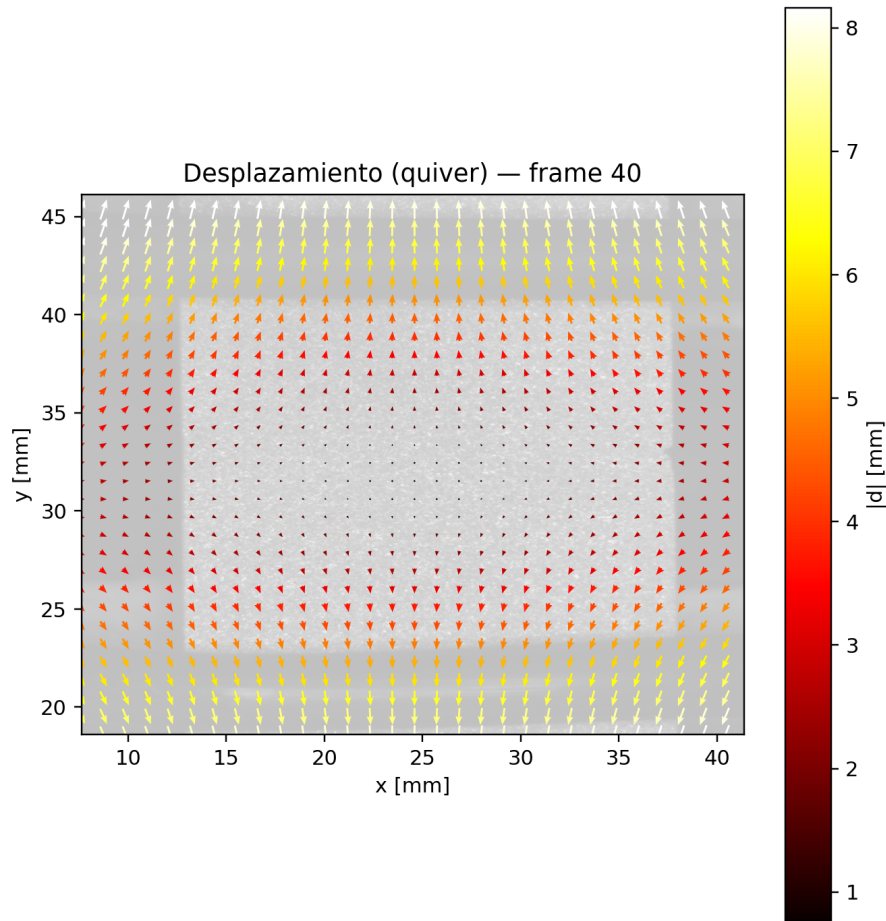


Figura 12: Campo vectorial de desplazamiento para el frame 40.

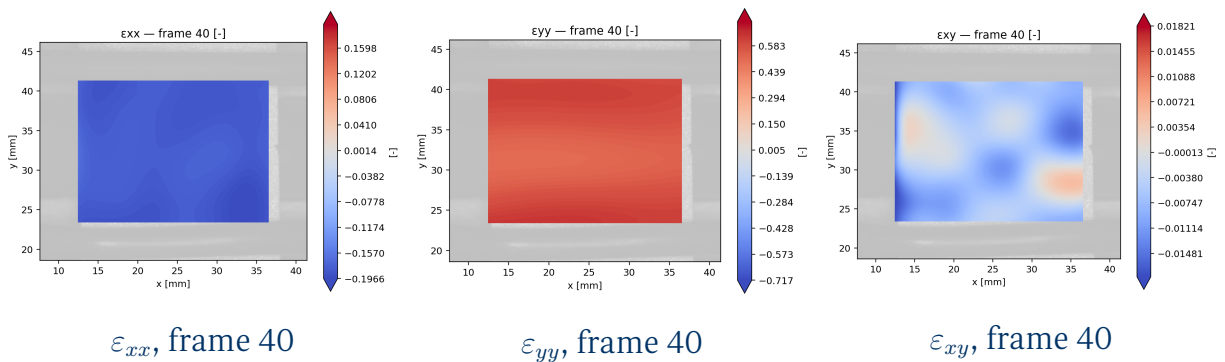


Figura 13: Mapas de deformación para el frame seleccionado. Permiten inspección puntual en cualquier instante del ensayo y validar que la reparación temporal de frames anómalos no introdujo artefactos en ese frame específico.

9 Archivos generados por el pipeline

El directorio `output_dir` contiene tres categorías de salidas.

9.1 Archivos de calibración

- `<project_name>_calibration_v3.npz`: calibración en formato binario NumPy. Contiene `mm_per_px`, `reprojection_error_px`, `stereo_rms_px`, `camera_matrix` y `dist_coeffs`.
- `<project_name>_calibration_v3.json`: el mismo contenido en formato texto JSON, legible sin Python.

9.2 Archivos de datos numéricos

9.2.1 Strain

- `strain_xx_v3.dat`, `strain_yy_v3.dat`, `strain_xy_v3.dat`: matrices $N_{\text{pts}} \times N_{\text{frames}}$ en formato texto delimitado por tabuladores, con valores en formato exponencial (`%.8e`).
- `strain_xx_v3_microstrain.dat`, etc.: mismas matrices multiplicadas por 10^6 , cuando `strain_display_units = "microstrain"`.

9.2.2 Incertidumbre

- `ux.dat`, `uy.dat`: incertidumbre estándar de U y V .
- `u_combined.dat`: incertidumbre combinada del vector de desplazamiento.
- `ustrain_xx.dat`, `ustrain_yy.dat`, `ustrain_xy.dat`: incertidumbre de las componentes del tensor.

9.3 Estado del proyecto

- `<project_name>.dicv3`: archivo de serialización completo del objeto `DICProjectV3`. Contiene todos los arrays de resultados y puede ser recuperado con `DICProjectV3.load()` para análisis posterior sin re-ejecutar el pipeline.

10 Criterios de configuración recomendados

Vínculo con Manual del Participante

Los rangos cuantitativos de los parámetros presentados en esta sección se corresponden con la Tabla de parámetros recomendados del §4.3.5 del Manual del Participante.

Parámetro	Efecto principal	Rango típico	Criterio de selección
subset_radius	Resolución espacial vs. ruido	10–40 px	$\geq$ 3–5 manchas del patrón
spacing_x/y	Densidad de grilla	5–30 px	$\leq$ subset_radius
search_radius	Rango de búsqueda	10–50 px	$>$ despl. máx. entre frames
corrcoef_min	Umbral de aceptación	0.4–0.7	Ajustar según calidad del patrón
strain_radius	VSG efectivo	3–15 nodos	$L = 2r\Delta >$ escala de interés
median_threshold	Sensibilidad al outlier	1.5–4.0	Menor = más restrictivo

Cuadro 2: Parámetros clave, su efecto y criterios de selección.

11 Limitaciones de la versión actual

Las siguientes características están declaradas en la configuración pero no están activas en V3.1:

1. **Aceleración GPU para correlación.** El parámetro `gpu_for_correlation` está presente pero el kernel NCC ejecuta en CPU. La aceleración GPU se aplica únicamente al remuestreo en visualización.
2. **Modelo de deformación afin del subset.** El parámetro `warp = "affine"` es declarativo. El motor actual implementa traslación pura. La extensión al modelo afin (6 parámetros: u, u_x, u_y, v, v_x, v_y) está reservada para versiones futuras.
3. **ROI poligonal interactiva.** El módulo `roi_grid.py` soporta ROI poligonal programática, pero el flujo predeterminado usa ROI rectangular interactiva.
4. **Convergencia iterativa IC-GN.** Los parámetros `cutoff_diffnorm` y `cutoff_iteration` están provistos para una implementación futura del optimizador IC-GN (*Inverse Compositional Gauss-Newton*). No están activos en el kernel NCC actual.